PERCEPTRON LEARNING RULE

VS

GRADIENT DESCENT

A Short Guide with Step-by-Step Mathematics

Author

Shaon Khan

BSc in ICT, IIT, Jahangirnagar University

Table of Contents

Contents

Introduction

This book explains two important learning rules in machine learning. The first is the Perceptron Learning Rule. The second is Gradient Descent.

Both methods update weights. But they update weights in different ways. This book shows the difference step by step.

We will use one small dataset. We will train it first with the Perceptron rule. Then we will train the same dataset again with Gradient Descent, so both methods can be compared on equal ground. After that, we will study a harder dataset, called XOR. We will see where the Perceptron fails. We will also see where a simple Gradient Descent model fails. Then we will see how a deep neural network, using activation functions and backpropagation, solves the problem.

At the end, three important questions are answered. A short Python program is also given. This program shows Gradient Descent in code.

Chapter 1: The Dataset

We start with a small dataset. It has two input features, x_1 and x_2. It has one output label, y. The label is either +1 or -1.

The Perceptron needs labels of +1 and -1. It does not use 0 and 1. The reason is explained later in this book.

x_1	x_2	y
1	2	+1
2	1	+1
3	2	-1
4	3	-1

This is our training data. We will use it again and again in this book.

Chapter 2: Initial Setup of the Perceptron

2.1 Starting Weights

We start all weights at zero. We also start the bias at zero.

$$w_1 = 0, \quad w_2 = 0, \quad b = 0$$

2.2 Learning Rate

The learning rate controls how big each update is. Here we choose a learning rate of 1.

$$\eta = 1$$

2.3 Activation Function

The Perceptron uses a step function. If the score z is zero or more, the output is +1. If the score z is less than zero, the output is -1.

$$\hat{y} = \{ +1 \ if z \geq 0 \qquad -1 \ if z < 0 \}$$

The score z is calculated as follows.

$$z = w_1 x_1 + w_2 x_2 + b$$

2.4 The Perceptron Loss Function

The Perceptron also has a loss function. A loss function is a number that tells us how wrong a prediction is. The Perceptron loss function is written below.

$$L = \Sigma \, max(0, -y(w^T x + b))$$

The term $w^T x + b$ is the same score we already called z. So the loss can also be written in a shorter form.

$$L = \Sigma \, max(0, -yz)$$

If a point is classified correctly, then y and z have the same sign. This means the product yz is positive. So -yz is negative. The maximum of 0 and a negative number is always 0. So a correctly classified point has zero loss.

If a point is classified wrong, then y and z have opposite signs. This means the product yz is negative. So -yz is positive. The maximum of 0 and a positive number is that positive number itself. So a wrongly classified point has a loss greater than 0.

This is exactly why the Perceptron only updates its weights for misclassified points. Correctly classified points already have zero loss. There is nothing to fix for those points.

Now we are ready to train the Perceptron, row by row.

Chapter 3: Training the Perceptron, Step by Step

We now go through each row of the dataset. For every row, we calculate the score z. Then we calculate the prediction. Then we check if the prediction is correct. If it is wrong, we update the weights and the bias.

3.1 Row 1: x = (1, 2), y = +1

First, we calculate the score.

$$z = (0)(1) + (0)(2) + 0 = 0$$

Since z is zero or more, the prediction is +1.

$$\hat{y} = +1$$

The true label is also +1. So the prediction is correct. No update is needed.

Weights stay the same:

$$w_1 = 0, \quad w_2 = 0, \quad b = 0$$

3.2 Row 2: x = (2, 1), y = +1

We calculate the score again, using the current weights.

$$z = (0)(2) + (0)(1) + 0 = 0$$

The prediction is +1.

$$\hat{y} = +1$$

The true label is +1. The prediction is correct. No update is needed.

Weights stay the same:

$$w_1 = 0, \quad w_2 = 0, \quad b = 0$$

3.3 Row 3: x = (3, 2), y = -1

We calculate the score.

$$z = (0)(3) + (0)(2) + 0 = 0$$

Since z is zero or more, the prediction is +1.

$$\hat{y} = +1$$

But the true label is -1. So the prediction is wrong. An update is needed.

Applying the Update Rule

The general update rule for the Perceptron is given below.

$$w = w + \eta\, y\, x$$

We apply this rule to each weight and to the bias.

Update for w_1:

$$w_1 = 0 + (1)(-1)(3) = -3$$

Update for w_2:

$$w_2 = 0 + (1)(-1)(2) = -2$$

Update for the bias:

$$b = 0 + (1)(-1) = -1$$

After this update, the new parameters are shown below.

$$w_1 = -3, \quad w_2 = -2, \quad b = -1$$

3.4 Row 4: x = (4, 3), y = -1

We calculate the score using the updated weights.

$$z = (-3)(4) + (-2)(3) + (-1) = -12 - 6 - 1 = -19$$

Since z is less than zero, the prediction is -1.

$$\hat{y} = -1$$

The true label is -1. The prediction is correct. No update is needed.

3.5 Final Parameters After One Epoch

After going through all four rows one time, this is called one epoch. The final parameters after this epoch are shown below.

$$w_1 = -3, \quad w_2 = -2, \quad b = -1$$

3.6 Training Summary Table

Row	Prediction	Correct?	Update?
1	+1	Yes	No
2	+1	Yes	No
3	+1	No	Yes
4	-1	Yes	No

Only one update happened in this epoch. This is because only one sample was misclassified. This is a key feature of the Perceptron. It only changes weights when it makes a mistake.

3.7 The Perceptron Convergence Theorem

There is a famous theorem about the Perceptron. It is called the Perceptron Convergence Theorem.

The theorem says the following. If the data is linearly separable, the Perceptron is guaranteed to converge. This means it will find a line that correctly separates every point, after a limited number of updates.

The dataset used in this book is linearly separable. This is why the Perceptron found a correct solution after only one update, as shown in this chapter. But if the data is not linearly separable, such as the XOR problem, this guarantee does not hold. The Perceptron may keep updating forever, and never find a stable solution. We will see this exact situation in a later chapter.

Chapter 4: Training with Gradient Descent, Step by Step

In this chapter, we train the same dataset again. But this time, we use Gradient Descent instead of the Perceptron rule. This lets us compare both methods on the same data.

4.1 The Mean Squared Error Loss Function

Gradient Descent needs a loss function that changes smoothly. In this chapter, we use the Mean Squared Error, or MSE, loss.

$$L = (1/n)\ \Sigma\ (\hat{y} - y)^2$$

Here, n is the number of training samples. ŷ is the predicted value. y is the true label. In this chapter, the model is a simple linear model. It does not use a step function.

$$\hat{y} = w_1 x_1 + w_2 x_2 + b$$

Unlike the Perceptron, this model gives a continuous output. It does not jump straight to +1 or -1.

4.2 Why Gradient Descent Uses the Derivative

The derivative of the loss function tells us the slope of the loss curve, at the current weights. This slope is called the gradient.

The gradient tells us which direction increases the loss. Gradient Descent moves in the opposite direction, to reduce the loss. This single idea is the core of the whole method.

$$w = w - \eta\ (\partial L / \partial w)$$

We now apply this idea to our dataset, one row at a time. We start with the same initial values as before.

$$w_1 = 0, \quad w_2 = 0, \quad b = 0, \quad \eta = 0.01$$

4.3 Row 1: x = (1, 2), y = +1

Prediction:

$$\hat{y} = (0)(1) + (0)(2) + 0 = 0$$

Error:

$$error = \hat{y} - y = 0 - 1 = -1$$

Squared error:

$$(error)^2 = (-1)^2 = 1$$

Gradient contribution from this row:

$$\partial L_1/\partial w_1 = 2(error)(x_1) = 2(-1)(1) = -2$$

$$\partial L_1/\partial w_2 = 2(error)(x_2) = 2(-1)(2) = -4$$

$$\partial L_1/\partial b = 2(error) = 2(-1) = -2$$

4.4 Row 2: x = (2, 1), y = +1

Prediction:

$$\hat{y} = (0)(2) + (0)(1) + 0 = 0$$

Error:

$$error = 0 - 1 = -1$$

Squared error:

$$(error)^2 = 1$$

Gradient contribution from this row:

$$\partial L_2/\partial w_1 = 2(-1)(2) = -4$$

$$\partial L_2/\partial w_2 = 2(-1)(1) = -2$$

$$\partial L_2/\partial b = 2(-1) = -2$$

4.5 Row 3: x = (3, 2), y = -1

Prediction:

$$\hat{y} = (0)(3) + (0)(2) + 0 = 0$$

Error:

$$error = 0 - (-1) = 1$$

Squared error:

$$(error)^2 = 1$$

Gradient contribution from this row:

$$\partial L_3/\partial w_1 = 2(1)(3) = 6$$

$$\partial L_3/\partial w_2 = 2(1)(2) = 4$$

$$\partial L_3/\partial b = 2(1) = 2$$

4.6 Row 4: x = (4, 3), y = -1

Prediction:

$$\hat{y} = (0)(4) + (0)(3) + 0 = 0$$

Error:

$$error = 0 - (-1) = 1$$

Squared error:

$$(error)^2 = 1$$

Gradient contribution from this row:

$$\partial L_4/\partial w_1 = 2(1)(4) = 8$$

$$\partial L_4/\partial w_2 = 2(1)(3) = 6$$

$$\partial L_4/\partial b = 2(1) = 2$$

4.7 Combining the Gradients and Updating the Parameters

We now collect the results from all four rows in one table.

Row	Error	Squared Error	dw_1	dw_2	db
1	-1	1	-2	-4	-2
2	-1	1	-4	-2	-2
3	1	1	6	4	2
4	1	1	8	6	2

First, we compute the loss for this epoch. This is the average of the squared errors.

$$L = (1 + 1 + 1 + 1) / 4 = 1$$

Next, we average the gradient contributions across all four rows.

$$\partial L/\partial w_1 = (-2 - 4 + 6 + 8) / 4 = 8 / 4 = 2$$

$$\partial L/\partial w_2 = (-4 - 2 + 4 + 6) / 4 = 4 / 4 = 1$$

$$\partial L/\partial b = (-2 - 2 + 2 + 2) / 4 = 0 / 4 = 0$$

Now we apply the Gradient Descent update rule to each parameter.

$$w_1 = 0 - (0.01)(2) = -0.02$$

$$w_2 = 0 - (0.01)(1) = -0.01$$

$$b = 0 - (0.01)(0) = 0$$

These are the new parameters, after one epoch of Gradient Descent.

$$w_1 = -0.02, \quad w_2 = -0.01, \quad b = 0$$

Notice something important here. The Perceptron changed its weights by large steps, such as -3 and -2, in a single update. Gradient Descent changes its weights by very small steps, such as -0.02 and -0.01. This happens because Gradient Descent updates using a small learning rate, and needs many epochs to reach good weights. The Python program later in this book runs 1000 such epochs, one after another.

Chapter 5: Perceptron Learning Rule vs Gradient Descent

The Perceptron rule and Gradient Descent are both used to update weights. But they work in different ways. We now compare them directly.

5.1 Method Comparison

Gradient Descent	Perceptron
Uses MSE or Cross-Entropy loss	Uses the Perceptron loss
Updates every iteration using gradients	Updates only when the prediction is wrong
Update: w = w - η(∂L/∂w)	Update: w = w + ηyx
Used for regression and deep learning	Used for binary classification

Gradient Descent always computes a gradient. It always takes a step, even if the step is very small. The Perceptron does not compute a gradient. It only reacts to mistakes.

5.2 How Gradient Descent Moves Toward the Minimum

Gradient Descent starts at some initial weight. It looks at the slope of the loss curve at that point. This slope is called the gradient. Then it takes a small step in the opposite direction of the gradient. It repeats this process again and again. Slowly, it moves down toward the lowest point of the loss curve. This lowest point is called the global cost minimum.

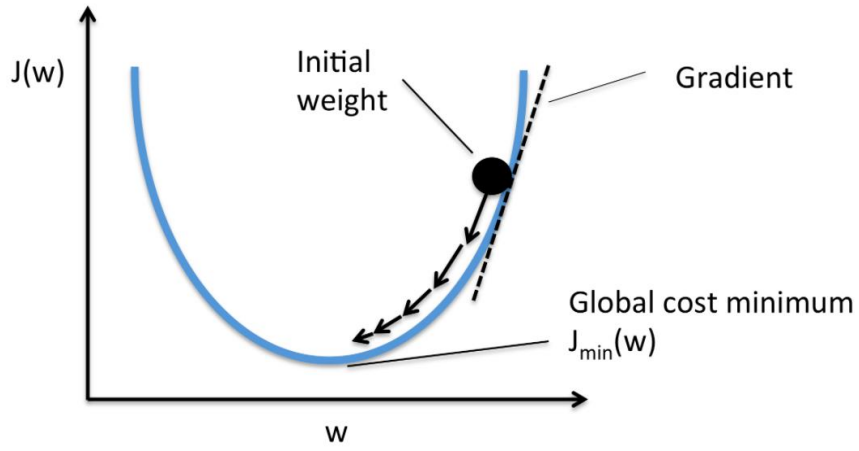

Figure 5.1: Gradient Descent moves the weight step by step toward the global cost minimum.

This is the normal, well-behaved case. But Gradient Descent does not always behave this well, as the next section explains.

5.3 Local Minimum vs Global Minimum

The loss curve in Figure 5.1 looks like a smooth bowl, with only one lowest point. This lowest point is the global minimum. It is the best possible set of weights for that loss function.

Real loss curves are not always this simple. Some loss curves have more than one valley. A point at the bottom of a small valley is called a local minimum. It is lower than its close neighbors, but it is not the lowest point of the entire curve.

Gradient Descent can sometimes get stuck moving between points on the loss curve, without ever settling at the true global minimum. Figure 5.2 shows this problem.

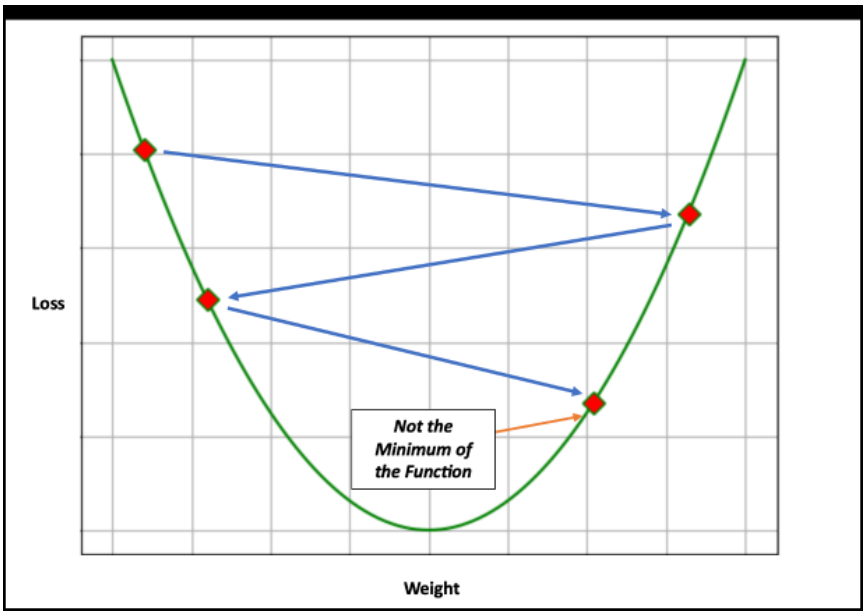

Figure 5.2: Gradient Descent can move between points on the loss curve without reaching the true minimum of the function.

This is one of the practical challenges of using Gradient Descent. Many modern training methods, such as momentum and adaptive learning rates, are designed to help avoid getting stuck this way.

5.4 Learning Rate: Overshooting and Slow Learning

The learning rate η controls the size of every update step. Choosing a good learning rate is important.

If the learning rate is too large, each update step becomes too big. The weights can jump past the minimum, and even land at a higher loss than before. This is called overshooting. In bad cases, the loss can keep growing, instead of getting smaller.

If the learning rate is too small, each update step becomes very small. The weights still move toward the minimum, but very slowly. Training can then take a very long time to finish.

A good learning rate is usually found through experiment. It should be small enough to avoid overshooting, but large enough to finish training in a reasonable time.

Chapter 6: Where Both Methods Fail — The XOR Problem

Many students believe that Gradient Descent can solve any problem that the Perceptron cannot solve. This is not correct. A single-layer model trained with Gradient Descent can also fail. The clearest example of this is the XOR problem.

6.1 The XOR Dataset

x_1	x_2	y

0	0	0
0	1	1
1	0	1
1	1	0

6.2 Why XOR Cannot Be Separated by One Line

If we place these four points on a graph, we see something important. The points with label 1 are at (0,1) and (1,0). The points with label 0 are at (0,0) and (1,1). No single straight line can separate these two groups. This is why XOR is called a problem that is not linearly separable.

6.3 The Perceptron on XOR

The Perceptron model is shown below.

$$\hat{y} = step(w_1x_1 + w_2x_2 + b)$$

The Perceptron keeps updating its weights using its own rule.

$$w = w + \eta(y - \hat{y})x$$

Suppose that, after some training, the weights become the following values.

$$w_1 = 1, \quad w_2 = 1, \quad b = -0.5$$

We check the predictions for all four points.

Input	Actual	Prediction	Correct?
(0, 0)	0	0	Yes
(0, 1)	1	1	Yes
(1, 0)	1	1	Yes
(1, 1)	0	1	No

The point (1, 1) is wrong. So the Perceptron updates its weights again to fix this point. But this new update breaks one of the other points that was correct before. This cycle repeats forever. The Perceptron never finds a stable line that separates all four points correctly.

Result: the Perceptron fails on XOR.

6.4 Gradient Descent with One Neuron on XOR

Now we replace the Perceptron rule with Gradient Descent. But we still use only one neuron. The model becomes a sigmoid model, shown below.

$$\hat{y} = \sigma(w_1 x_1 + w_2 x_2 + b)$$

The loss function used here is called binary cross-entropy loss.

$$L = -\Sigma[\, y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}) \,]$$

Gradient Descent updates the weights using this rule.

$$w = w - \eta(\partial L / \partial w)$$

We repeat this update for many iterations. The loss slowly goes down. But it never reaches zero. The accuracy stays around 50 to 75 percent.

The reason is simple. Even with Gradient Descent, the score inside the sigmoid is still just one straight line.

$$z = w_1 x_1 + w_2 x_2 + b$$

One straight line cannot separate the XOR points. This is true no matter how well Gradient Descent tunes the weights of this single line, and no matter how many epochs it runs. So Gradient Descent, when used with only one neuron, also fails on XOR.

Result: Gradient Descent with a single neuron fails on XOR.

6.5 Deep Learning Solves XOR

Now we add one hidden layer. This hidden layer sits between the input and the output.

x_1 and x_2 connect to several hidden neurons. These hidden neurons connect to the output neuron.

This small change makes the model non-linear. Gradient Descent, combined with backpropagation, now updates every weight in the network, including the hidden layer.

After enough training, the network produces the correct output for every input.

Input	Actual	Prediction	Correct?
(0, 0)	0	0	Yes
(0, 1)	1	1	Yes
(1, 0)	1	1	Yes
(1, 1)	0	0	Yes

The accuracy is now 100 percent. Result: a deep network with hidden layers passes on XOR.

6.6 Final Comparison Table

Method	Result on XOR
Perceptron Learning Rule	Fail
Gradient Descent with a Single Neuron	Fail
Gradient Descent with a Multi-Layer Network	Pass

At first glance, the Perceptron Learning Rule and Gradient Descent look almost the same because both change the model's weights to improve prediction. However, they are not the same method. The Perceptron Learning Rule is a special learning rule designed only for the perceptron. It checks whether a prediction is correct or wrong. If the prediction is correct, it does nothing. If the prediction is wrong, it immediately updates the weights using the input and the class label. Gradient Descent works in a different way. It first calculates a loss value, then computes the gradient, which tells how the loss changes with each weight. After that, it updates the weights by moving in the direction that reduces the loss. This update happens every iteration, even if the prediction is already close to correct. In simple words, the Perceptron learns only from mistakes, while Gradient Descent learns by continuously reducing the loss. Although the Perceptron update can be written as a Gradient Descent step on the perceptron loss, Gradient Descent is a general optimization method used in many machine learning and deep learning models, whereas the Perceptron Learning Rule is a special case designed for the perceptron. This distinction is the real difference between them.

Chapter 7: Activation Functions in Neural Networks

Neural networks use activation functions inside their neurons. An activation function decides the final output of a neuron, based on its input score z. Different activation functions are used for different purposes.

7.1 Step Function

The step function is the one used by the Perceptron in this book. It outputs +1 or -1, based on whether z is positive or negative.

$$step(z) = \{ +1 \ if z \geq 0 \qquad -1 \ if z < 0 \}$$

This function has a sharp jump at zero, and no smooth slope anywhere else. This is why it cannot be used with Gradient Descent, which needs a smooth derivative at every point.

7.2 Sigmoid Function

The sigmoid function produces a smooth output, between 0 and 1.

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

This function is used in the single-neuron model tested on the XOR problem, earlier in this book. Its output can be understood as a probability between 0 and 1.

7.3 Tanh Function

The tanh function is similar to sigmoid in shape, but its output ranges from -1 to +1, instead of 0 to 1.

$$tanh(z) = (e^z - e^(-z)) / (e^z + e^(-z))$$

Because tanh is centered at zero, it can make training easier in some networks, compared to sigmoid.

7.4 ReLU Function

ReLU stands for Rectified Linear Unit. It is one of the most common activation functions used in deep learning today.

$$ReLU(z) = max(0, z)$$

ReLU outputs z directly when z is positive, and outputs 0 when z is negative. It is simple and fast to compute, and it works well in deep networks with many layers.

These activation functions are placed inside the hidden neurons of a deep network, such as the one that solved the XOR problem in Chapter 6. Without a non-linear activation function like sigmoid, tanh, or ReLU, a hidden layer would behave just like a single straight line, and the network would not be able to solve problems like XOR.

Chapter 8: Forward Pass, Loss, and Backpropagation

Training a deep neural network involves four repeating steps. These steps are the forward pass, the loss calculation, backpropagation, and the Gradient Descent update. This chapter explains each step in simple terms.

8.1 Forward Pass

In the forward pass, the input values move through the network, from the input layer toward the output layer. Each neuron computes its score z, applies an activation function, and passes its output to the next layer. At the end of the forward pass, the network produces a prediction ŷ.

8.2 Loss

The loss step compares the prediction ŷ with the true label y. It produces a single number that tells us how wrong the prediction is. A small loss means the prediction is close to the true label. A large loss means the prediction is far from the true label.

8.3 Backpropagation

Backpropagation moves in the opposite direction of the forward pass. It starts at the output layer, and moves backward, toward the input layer. At each layer, it uses a rule called the chain rule, to calculate the gradient of the loss, with respect to every weight in that layer.

Backpropagation is important because a deep network can have thousands, or even millions, of weights. Without backpropagation, it would be far too slow to calculate every gradient separately. Backpropagation calculates all of them efficiently, in a single backward pass through the network.

8.4 Gradient Descent Update

Once every gradient is known, Gradient Descent updates every weight in the network, using the same rule we used earlier in this book.

$$w = w - \eta \, (\partial L / \partial w)$$

These four steps, forward pass, loss, backpropagation, and the Gradient Descent update, repeat again and again, for many epochs. This is how a deep neural network learns to solve problems like XOR, as shown in Chapter 6.

Chapter 9: The Real Reason Gradient Descent Solves XOR

Many students think that Gradient Descent solved XOR because it is a smarter algorithm than the Perceptron rule. This idea is not correct.

The true reason is different. The Perceptron rule only works with a single layer. Gradient Descent, by itself, is only a general optimization method. It can be used with a single neuron or with many layers.

Gradient Descent does not solve XOR by itself. It solves XOR only when it is combined with hidden layers and backpropagation. Hidden layers, together with non-linear activation functions, make the model non-linear. Backpropagation allows Gradient Descent to update every layer of the network.

So the correct summary is this: backpropagation, together with Gradient Descent, hidden layers, and non-linear activation functions, is what allows deep learning to solve problems like XOR. Gradient Descent alone is not enough.

Chapter 10: Questions and Answers

Q1. Why does the Perceptron use labels -1 and +1, instead of 0 and 1?

The Perceptron uses -1 and +1 because these two numbers represent two opposite classes in a clean and symmetric way. This choice makes the update rule simple.

The update rule is shown below.

$$w = w + \eta yx$$

If y is +1, the weights move toward the input vector x. If y is -1, the weights move away from the input vector x.

If we used labels of 0 and 1 instead, there would be a problem. When y is 0, the update term becomes zero.

$$\eta \times 0 \times x = 0$$

This means the weights would not change, even when the prediction was wrong. So the labels -1 and +1 make the update rule work correctly for both classes.

Q2. Gradient Descent subtracts the gradient. Why does the Perceptron add the term ηyx instead?

Gradient Descent updates weights using this rule.

$$w = w - \eta(\partial L/\partial w)$$

It subtracts the gradient because this direction reduces the loss.

The Perceptron update rule is different in appearance.

$$w = w + \eta yx$$

But this is only a different appearance. For a misclassified point, the gradient of the Perceptron loss function is negative of yx.

$$\partial L/\partial w = -yx$$

If we place this gradient into the Gradient Descent formula, we get the following result.

$$w = w - \eta(-yx) = w + \eta yx$$

So the Perceptron update rule is really the same as a Gradient Descent step. It only looks different because the gradient itself already contains a negative sign.

Q3 (Optional). How is the Perceptron rule a form of Gradient Descent? What is one limitation of the Perceptron loss?

The Perceptron learning rule can be seen as a special case of Gradient Descent. It minimizes the Perceptron loss function shown earlier in this book. The weight update comes from taking a Gradient Descent step on this loss function.

The difference is that the Perceptron only updates its weights for misclassified points. A correctly classified point produces zero loss. So it produces no update.

One limitation of the Perceptron loss is that it only works when the data can be separated by a single straight line. If the classes cannot be separated this way, such as in the XOR problem, the Perceptron does not converge. It never finds a correct and stable solution.

Modern neural networks solve this limitation. They use smoother loss functions, such as cross-entropy loss. They also use hidden layers, non-linear activation functions, backpropagation, and Gradient Descent, all together.

Chapter 11: Python Code for Gradient Descent

Below is a small scratch program written in plain Python. It does not use any machine learning library. It shows Gradient Descent for a simple linear model, trained with mean squared error loss, matching the hand calculation shown in Chapter 4.

```
import numpy as np

# ------------------------------------------------
# Simple Gradient Descent from scratch
# Model: y_hat = w1*x1 + w2*x2 + b
# Loss  : Mean Squared Error (MSE)
# ------------------------------------------------

# Training data (x1, x2, y)
X = np.array([
    [1, 2],
    [2, 1],
    [3, 2],
    [4, 3]
], dtype=float)

y = np.array([1, 1, -1, -1], dtype=float)

# Initialize weights and bias at zero
w1, w2, b = 0.0, 0.0, 0.0

# Learning rate and number of iterations
eta = 0.01
epochs = 1000
n = len(X)
```

```
for epoch in range(epochs):
    total_loss = 0.0
    dw1 = dw2 = db = 0.0

    for i in range(n):
        x1, x2 = X[i]
        y_true = y[i]

        # Forward pass: compute prediction
        z = w1 * x1 + w2 * x2 + b
        y_hat = z

        # Error for this sample
        error = y_hat - y_true
        total_loss += error ** 2

        # Gradients of MSE loss
        dw1 += 2 * error * x1
        dw2 += 2 * error * x2
        db  += 2 * error

    # Average the gradients over all samples
    dw1 /= n
    dw2 /= n
    db  /= n

    # Gradient Descent update rule
    w1 = w1 - eta * dw1
    w2 = w2 - eta * dw2
    b  = b  - eta * db

    if epoch % 100 == 0:
        avg_loss = total_loss / n
        print(f"Epoch {epoch}: Loss = {avg_loss:.4f}, "
              f"w1 = {w1:.4f}, w2 = {w2:.4f}, b = {b:.4f}")

print("\nFinal parameters:")
print(f"w1 = {w1:.4f}, w2 = {w2:.4f}, b = {b:.4f}")
```

This program shows the two main steps of Gradient Descent. First, it computes the gradient of the loss for every weight, exactly as we did by hand in Chapter 4. Second, it moves each weight a small step in the opposite direction of its gradient. It repeats this process for many epochs, until the loss becomes small.

Conclusion

This book compared two learning rules. The Perceptron rule updates weights only when it makes a mistake. Gradient Descent updates weights on every iteration, using a gradient.

We trained a small dataset with the Perceptron rule, and then with Gradient Descent, and saw exactly how each weight changed in both cases. We studied local and global minima, and the effect of the learning rate. We then studied the XOR problem. We saw that both the Perceptron rule and a single-neuron Gradient Descent model fail on this problem. Only a deep network, with hidden layers, non-linear activation functions, and backpropagation, can solve it.

The key lesson is this. Gradient Descent is a general tool for optimization. It is not, by itself, a solution to non-linear problems. Hidden layers, activation functions, and backpropagation are what give deep learning its real power.